Cache Sizes (Zedboard)
Cortex-A9 MPCore - per core…

- 32 KB L1 data + 32 KB L1 instructin
- both cores share 512 KB L2
- no L3

Any tiling must remain within 32 KB for L1 or within 512 KB for L2

CPU

```
bash-5.1$ lscpu | egrep 'Model name|L1d|L2|L3'
getconf LEVEL1_DCACHE_SIZE; getconf LEVEL2_CACHE_SIZE; getconf LEVEL3_CACHE_SIZE
# Or: cat /sys/devices/system/cpu/cpu0/cache/index*/{type,level,size}
Model name:                          Intel(R) Xeon(R) Gold 6354 CPU @ 3.00GHz
L1d cache:                           256 KiB (8 instances)
L2 cache:                            32 MiB (8 instances)
L3 cache:                            16 MiB (1 instance)
32768
2097152
16777216
```

Slowest Layer (reuse goal)
In the slowest layer (Convolution), the triple loop over output * filters * input depth repeatedly streams through weights and activations with no reuse. The goal is to reorder the indexes and the layout so chunks of weights and inputs are reused to generate several outputs before moving on.

Tiling

- Break loops into blocks whose combined footprint of input tile, filter tile, and output tile fits in the cache
- Process one tile fully, then advance to the next
- Improving temporal and spatial locality for all three tensors

Identifying Memory Access Patterns

- Inputs are stored NHWC. Naive slides a 2-D window one pixel at a time, so you get good spatial locality along x and then y, but every output channel re-reads the same path
- Weights have output channel m as the first loop, so each weight slice reused across many output positions
- Because we accumulate one channel at a time, the write-allocate lines hang around while we’re still recomputing other channels
- Without blocking, the same input chunk is fetched repeatedly for different output channels and accumulators

Speedup from Optimization

```
bash-5.1$ cd cpre*
bash-5.1$ make
bash-5.1$ ./build/ml
[Info]: --- Building Toy Model ---
Opened binary file data_fp32/model/conv1_weights.bin
Opened binary file data_fp32/model/conv1_biases.bin
Opened binary file data_fp32/model/conv2_weights.bin
Opened binary file data_fp32/model/conv2_biases.bin
Opened binary file data_fp32/model/conv3_weights.bin
Opened binary file data_fp32/model/conv3_biases.bin
Opened binary file data_fp32/model/conv4_weights.bin
Opened binary file data_fp32/model/conv4_biases.bin
Opened binary file data_fp32/model/conv5_weights.bin
Opened binary file data_fp32/model/conv5_biases.bin
Opened binary file data_fp32/model/conv6_weights.bin
Opened binary file data_fp32/model/conv6_biases.bin
Opened binary file data_fp32/model/dense1_weights.bin
Opened binary file data_fp32/model/dense1_biases.bin
Opened binary file data_fp32/model/dense2_weights.bin
Opened binary file data_fp32/model/dense2_biases.bin
[Info]: --- Running Layer Test 0 ---
Opened binary file data_fp32/image_0.bin
Timer Layer Inference: elapsed=8.602000ms
Opened binary file data_fp32/image_0_data/layer_0_output.bin
[Info]: Naive vs expected (layer 0)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 0)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 0)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=507.221008ms
Timer Layer Benchmark: elapsed=98.585999ms
[Info]: Benchmark (layer 0): Naive 5.072210 ms vs Tiled 0.985860 ms -> 5.144960x
[Info]: --- Running Layer Test 1 ---
Opened binary file data_fp32/image_0_data/layer_0_output.bin
Timer Layer Inference: elapsed=49.043999ms
Opened binary file data_fp32/image_0_data/layer_1_output.bin
[Info]: Naive vs expected (layer 1)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 1)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 1)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=4886.796875ms
Timer Layer Benchmark: elapsed=740.114014ms
[Info]: Benchmark (layer 1): Naive 48.867970 ms vs Tiled 7.401140 ms -> 6.602762x
[Info]: --- Running Layer Test 2 ---
Opened binary file data_fp32/image_0_data/layer_1_output.bin
Timer Layer Inference: elapsed=0.156000ms
Opened binary file data_fp32/image_0_data/layer_2_output.bin
[Info]: Naive vs expected (layer 2)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 2)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 2)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=13.836000ms
Timer Layer Benchmark: elapsed=9.850000ms
[Info]: Benchmark (layer 2): Naive 0.138360 ms vs Tiled 0.098500 ms -> 1.404670x
```

```
[Info]: --- Running Layer Test 3 ---
Opened binary file data_fp32/image_0_data/layer_2_output.bin
Timer Layer Inference: elapsed=9.010000ms
Opened binary file data_fp32/image_0_data/layer_3_output.bin
[Info]: Naive vs expected (layer 3)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 3)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 3)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=887.318970ms
Timer Layer Benchmark: elapsed=116.795998ms
[Info]: Benchmark (layer 3): Naive 8.873190 ms vs Tiled 1.167960 ms -> 7.597170x
[Info]: --- Running Layer Test 4 ---
Opened binary file data_fp32/image_0_data/layer_3_output.bin
Timer Layer Inference: elapsed=15.544000ms
Opened binary file data_fp32/image_0_data/layer_4_output.bin
[Info]: Naive vs expected (layer 4)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 4)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 4)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=1543.792969ms
Timer Layer Benchmark: elapsed=191.110001ms
[Info]: Benchmark (layer 4): Naive 15.437930 ms vs Tiled 1.911100 ms -> 8.078033x
[Info]: --- Running Layer Test 5 ---
Opened binary file data_fp32/image_0_data/layer_4_output.bin
Timer Layer Inference: elapsed=0.049000ms
Opened binary file data_fp32/image_0_data/layer_5_output.bin
[Info]: Naive vs expected (layer 5)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 5)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 5)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=3.903000ms
Timer Layer Benchmark: elapsed=3.594000ms
[Info]: Benchmark (layer 5): Naive 0.039030 ms vs Tiled 0.035940 ms -> 1.085977x
[Info]: --- Running Layer Test 6 ---
Opened binary file data_fp32/image_0_data/layer_5_output.bin
Timer Layer Inference: elapsed=2.720000ms
Opened binary file data_fp32/image_0_data/layer_6_output.bin
[Info]: Naive vs expected (layer 6)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 6)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 6)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=271.322998ms
Timer Layer Benchmark: elapsed=43.602001ms
[Info]: Benchmark (layer 6): Naive 2.713230 ms vs Tiled 0.436020 ms -> 6.222719x
```

```
[Info]: --- Running Layer Test 7 ---
Opened binary file data_fp32/image_0_data/layer_6_output.bin
Timer Layer Inference: elapsed=3.422000ms
Opened binary file data_fp32/image_0_data/layer_7_output.bin
[Info]: Naive vs expected (layer 7)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 7)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 7)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=339.256012ms
Timer Layer Benchmark: elapsed=52.532001ms
[Info]: Benchmark (layer 7): Naive 3.392560 ms vs Tiled 0.525320 ms -> 6.458083x
[Info]: --- Running Layer Test 8 ---
Opened binary file data_fp32/image_0_data/layer_7_output.bin
Timer Layer Inference: elapsed=0.012000ms
Opened binary file data_fp32/image_0_data/layer_8_output.bin
[Info]: Naive vs expected (layer 8)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 8)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 8)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=0.775000ms
Timer Layer Benchmark: elapsed=0.788000ms
[Info]: Benchmark (layer 8): Naive 0.007750 ms vs Tiled 0.007880 ms -> 0.983503x
[Info]: --- Running Layer Test 9 ---
Opened binary file data_fp32/image_0_data/layer_8_output.bin
Timer Layer Inference: elapsed=0.001000ms
Opened binary file data_fp32/image_0_data/layer_9_output.bin
[Info]: Naive vs expected (layer 9)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 9)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 9)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=0.015000ms
Timer Layer Benchmark: elapsed=0.027000ms
[Info]: Benchmark (layer 9): Naive 0.000150 ms vs Tiled 0.000270 ms -> 0.555555x
[Info]: --- Running Layer Test 10 ---
Opened binary file data_fp32/image_0_data/layer_9_output.bin
Timer Layer Inference: elapsed=1.119000ms
Opened binary file data_fp32/image_0_data/layer_10_output.bin
[Info]: Naive vs expected (layer 10)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs expected (layer 10)
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Tiled vs Naive (layer 10)
Comparing Outputs (Cosine Similarity): True 100%  (1)
Timer Layer Benchmark: elapsed=91.362000ms
Timer Layer Benchmark: elapsed=20.295000ms
[Info]: Benchmark (layer 10): Naive 0.913620 ms vs Tiled 0.202950 ms -> 4.501700x
```

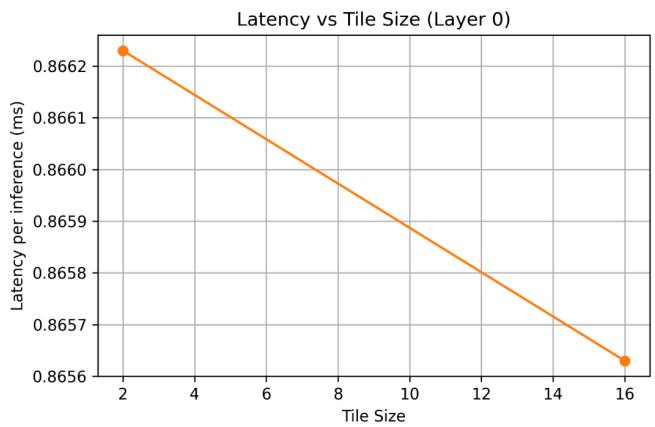
As seen in the inference tests, Tiling works as expected, creating much more data reuse per layer.

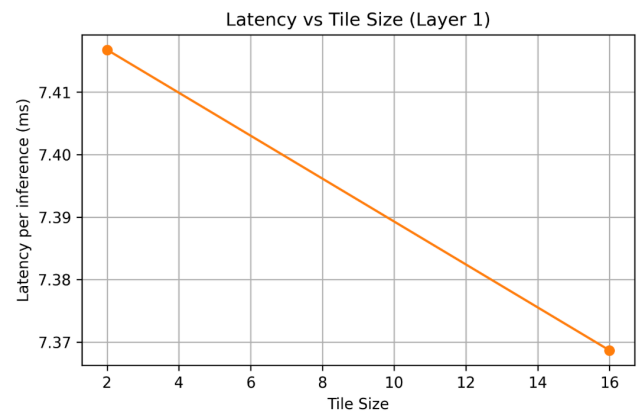
Seepedups across layers from Naive -> Tiling are displayed

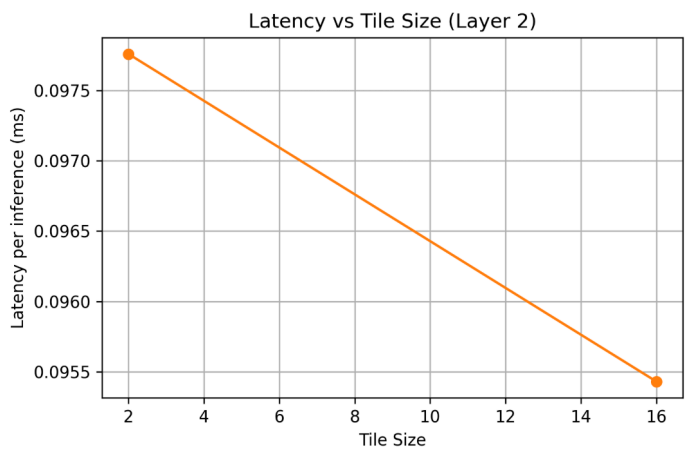
We can see that layers without tiling implementation stay similar in speed (Softmax, Flatten) while layers with Tiling implementation gain massive speedups simply from going over fewer input chunks - instead we reuse them for multiple output channels and accumulators

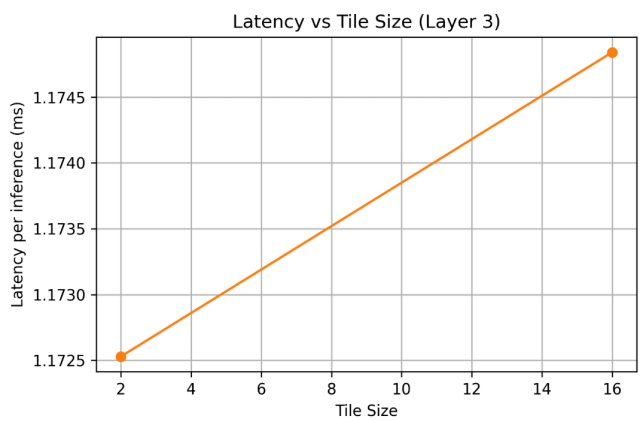
Access Patterns Graphs

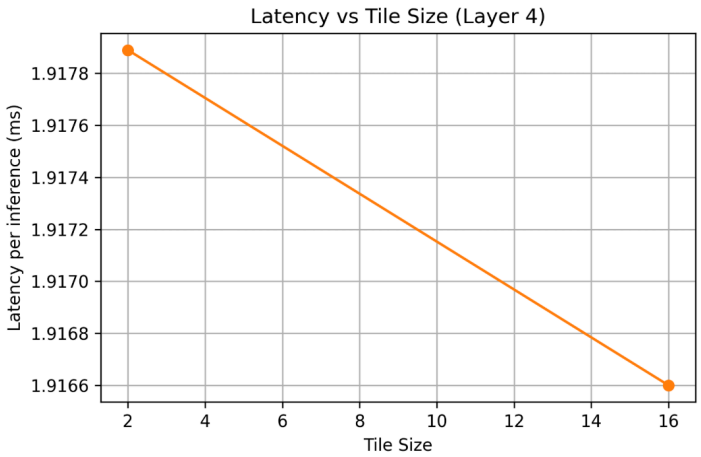
```
Performance counter stats for './build/ml':

       3229189511      cpu_atom/L1-dcache-loads/                                          (0.02%)
      64493946115      cpu_core/L1-dcache-loads/                                          (99.98%)
   <not supported>     cpu_atom/L1-dcache-load-misses/
      11256764163      cpu_core/L1-dcache-load-misses/  #   17.45% of all L1-dcache accesses   (99.98%)
       1444895859      cpu_atom/L1-dcache-stores/                                         (0.02%)
       6075937232      cpu_core/L1-dcache-stores/                                         (99.98%)
   <not supported>     cpu_atom/L1-dcache-store-misses/
   <not supported>     cpu_core/L1-dcache-store-misses/

     10.530104719 seconds time elapsed

     10.499639000 seconds user
      0.004965000 seconds sys
```

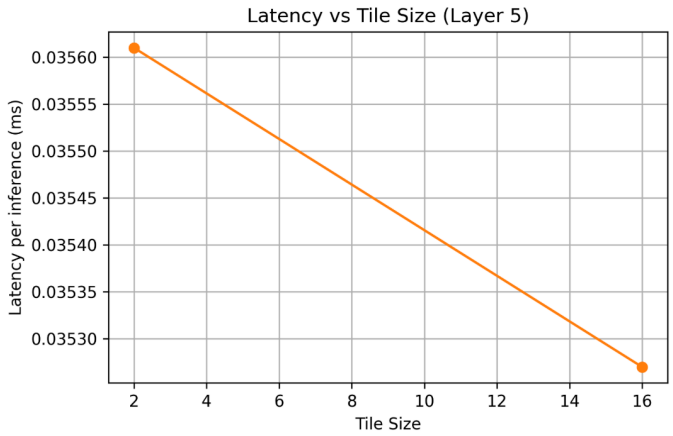


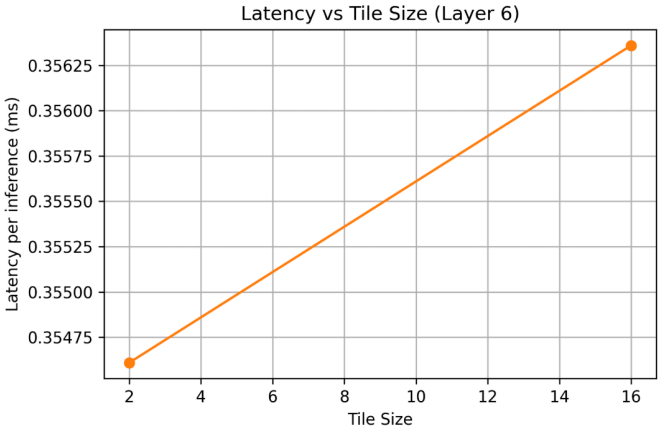


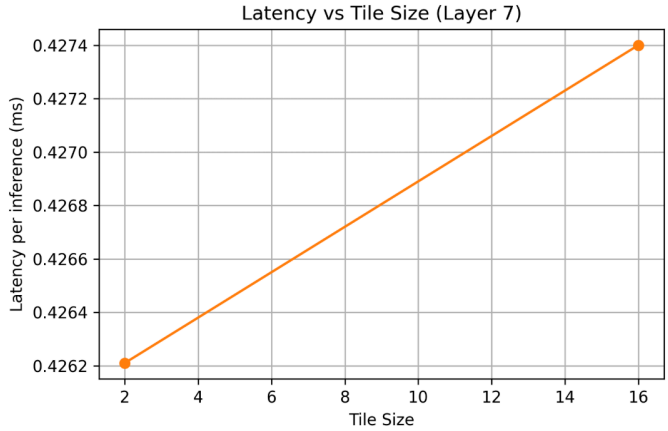


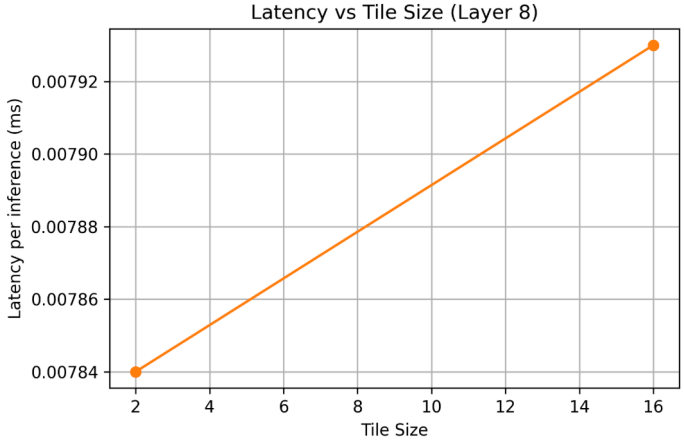


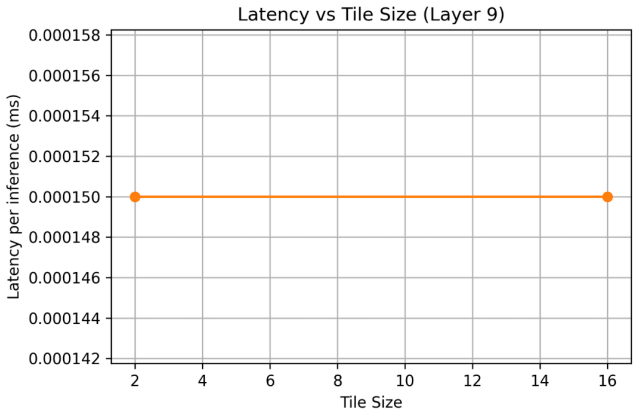

Latency vs Tile Size (Layer 4)
Latency per inference (ms)
1.9178
1.9176
1.9174
1.9172
1.9170
1.9168
1.9166
2 4 6 8 10 12 14 16
Tile Size

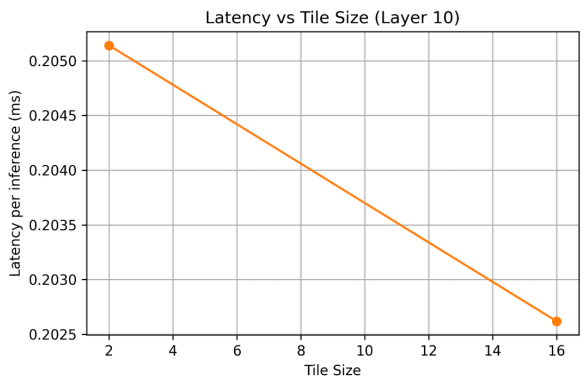

Latency vs Tile Size (Layer 5)
Latency per inference (ms)
0.03560
0.03555
0.03550
0.03545
0.03540
0.03535
0.03530
2 4 6 8 10 12 14 16
Tile Size


Latency vs Tile Size (Layer 6)
Latency per inference (ms)
0.35625
0.35600
0.35575
0.35550
0.35525
0.35500
0.35475
2 4 6 8 10 12 14 16
Tile Size


Latency vs Tile Size (Layer 7)
Latency per inference (ms)
0.4274
0.4272
0.4270
0.4268
0.4266
0.4264
0.4262
2 4 6 8 10 12 14 16
Tile Size


Latency vs Tile Size (Layer 8)
Latency per inference (ms)
0.00792
0.00790
0.00788
0.00786
0.00784
2 4 6 8 10 12 14 16
Tile Size


Latency vs Tile Size (Layer 9)
Latency per inference (ms)
0.000158
0.000156
0.000154
0.000152
0.000150
0.000148
0.000146
0.000144
0.000142
2 4 6 8 10 12 14 16
Tile Size

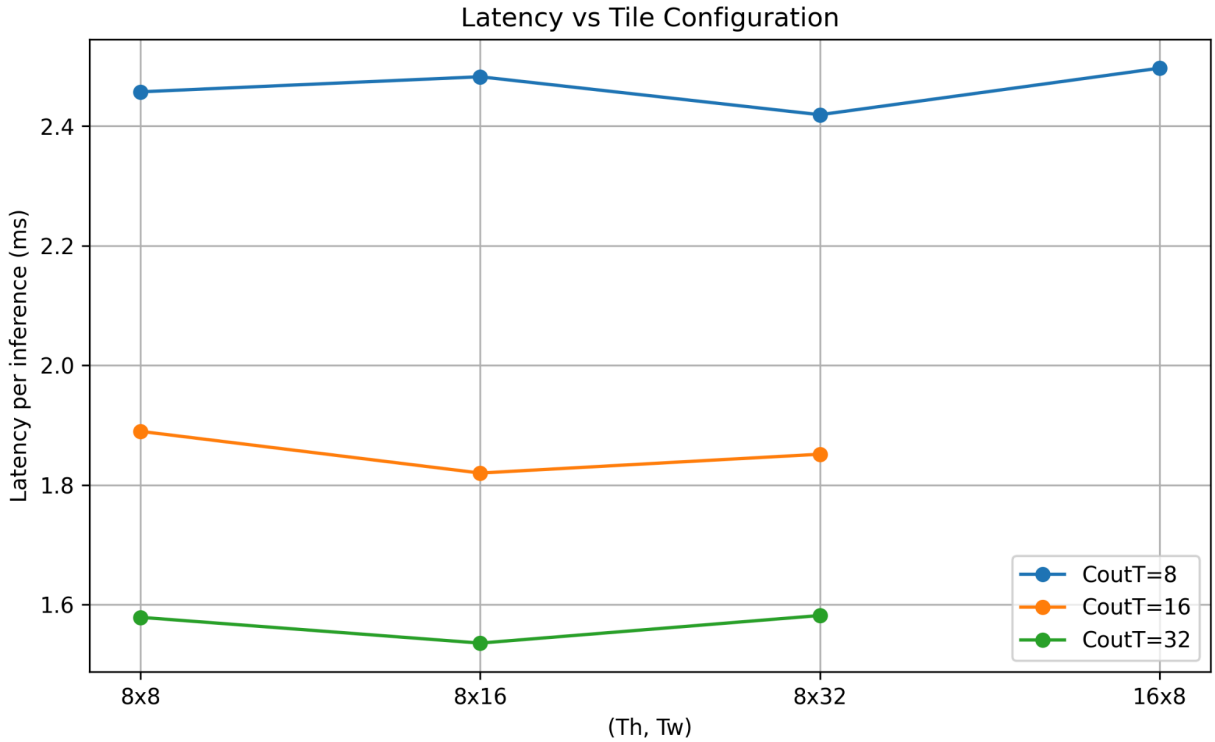


Tile Size constant set constant at 16 for this experiment
Values in table show avg values over 100 runs

	Avg Naive(ms)	Avg Tiling(ms)	Speedup
Layer 0	5.102240	0.865630	5.894250
Layer 1	48.998692	7.368680	6.649589
Layer 2	0.134490	0.095430	1.409305
Layer 3	8.908040	1.174840	7.582343
Layer 4	15.585050	1.916600	8.131613
Layer 5	0.039530	0.035270	1.120783
Layer 6	2.652240	0.356360	7.442586
Layer 7	3.391200	0.427400	7.934487
Layer 8	0.007550	0.007930	0.952081
Layer 9	0.000130	0.000150	0.866667
Layer 10	0.880090	0.202620	4.343550

Total L1 Loads(avg across 100 runs - all layers): 64,479,678,062
Total L1 Misses: 11,199,643,768
L1 miss percentage: 17.369%

Varied Tile Size Graph




```
Th,Tw,CoutT,time_ms,l1_miss_rate,cache_miss_rate
8,8,8,2.457110,18.099738,0.134515
8,16,8,2.482330,18.526882,0.160663
8,32,8,2.418830,19.164087,0.156696
16,8,8,2.496720,18.544288,0.888071
8,8,16,1.889530,20.092768,0.148826
8,16,16,1.819800,20.054033,0.158056
8,32,16,1.851300,20.979170,0.218809
8,8,32,1.578650,20.898180,0.153980
8,16,32,1.535500,21.227632,0.211101
8,32,32,1.581650,22.226176,0.181461
```

Ran Multiple runs comparing Latency vs Tile Size - ran 1 inference for TileSize = 8, which explains the extra plot

As shown in the graph, the larger tile size(32) fits greater amounts of data in the cache at a time and has the lowest latency, allowing for the maximum data reuse.

L1 Miss rates stay relatively consistent

Tiling Zedboard Results (See test times with SIMD)

Layer	Zedboard Naive (ms)	Zedboard Tiled (ms)	Speedup
0	101	144	**0.70x slower** - bad tile fit
1	1878	1324	**1.42x faster**
2	5	3	**1.67x faster**
3	293	207	**1.42x faster**
4	506	349	**1.45x faster**
5	1	1	**Same -** too small to matter
6	87	61	**1.43x faster**
7	112	77	**1.43x faster**
8	Max pool	Max pool	n/a

Layer	CPU Naive (ms)	CPU Tiled (ms)	CPU Speedup
0	5.102	0.866	**5.89x**
1	48.999	7.369	**6.65x**
2	0.134	0.095	**1.41x**
3	8.908	1.175	**7.58x**
4	15.585	1.917	**8.13x**
5	0.039	0.035	**1.12x**
6	2.652	0.356	**7.44x**
7	3.391	0.427	**7.93x**
8	0.00755	0.00793	**0.95x (noise level)**
9	0.000013	0.00015	**0.86x (noise level)**

Conv layer	CPU Tiled (ms)	Zed Tiled (ms)	Zed Speed (slower obv)
0	0.866	144	166x slowdown
1	7.369	1324	180x
2	0.095	3	31x
3	1.175	207	176x
4	1.917	349	182x
6	0.356	61	171x
7	0.427	77	180x

Larger output tiles cut loop overhead, but they also grow the active cache set. Once the tile "footprint" exceeds L1/L2 capacity or maps poorly to the cache's sets, lines get removed before reused. That forces each revisit to reload data, so total cache misses rise even though you're doing fewer outer-loop iterations
Bigger tiles improve arithmetic intensity only while the tile fits comfortably in cache; after that point, capacity/conflict pressure dominates and the "more reuse per tile" benefit is overwhelmed by evictions

SIMD Implementation (SSE2)
Uses 128-bit vectors

Layer 0	36ms
1	487ms
2	5ms
3	73ms
4	118ms
5	1ms
6	20ms
7	30ms
8	<1ms

9	<1ms
10	8ms
11	<1ms
Full Inference	783ms

```
xsdb% [Info]: --- Running Layer Test 0 ---
xsdb% Opened binary file data/image_0.bin
xsdb% Opened binary file data/image_0_data/layer_0_output.bin
xsdb% Timer Layer Naive: elapsed=101.000000ms
xsdb% Timer Layer SIMD: elapsed=36.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 0 timings (ms): naive=101.00, simd=36.00, speedup=2.81x
[Info]: --- Running Layer Test 1 ---
xsdb% Opened binary file data/image_0_data/layer_0_output.bin
xsdb% Opened binary file data/image_0_data/layer_1_output.bin
xsdb% Timer Layer Naive: elapsed=1877.000000ms
xsdb% Timer Layer SIMD: elapsed=487.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 1 timings (ms): naive=1877.00, simd=487.00, speedup=3.85x
[Info]: --- Running Layer Test 2 ---
xsdb% Opened binary file data/image_0_data/layer_1_output.bin
xsdb% Opened binary file data/image_0_data/layer_2_output.bin
xsdb% Timer Layer Naive: elapsed=5.000000ms
xsdb% Timer Layer SIMD: elapsed=5.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 2 timings (ms): naive=5.00, simd=5.00, speedup=1.00x
[Info]: --- Running Layer Test 3 ---
xsdb% Opened binary file data/image_0_data/layer_2_output.bin
xsdb% Opened binary file data/image_0_data/layer_3_output.bin
xsdb% Timer Layer Naive: elapsed=293.000000ms
xsdb% Timer Layer SIMD: elapsed=73.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 3 timings (ms): naive=293.00, simd=73.00, speedup=4.01x
[Info]: --- Running Layer Test 4 ---
xsdb% Opened binary file data/image_0_data/layer_3_output.bin
xsdb% Opened binary file data/image_0_data/layer_4_output.bin
xsdb% Timer Layer Naive: elapsed=506.000000ms
xsdb% Timer Layer SIMD: elapsed=118.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 4 timings (ms): naive=506.00, simd=118.00, speedup=4.29x
[Info]: --- Running Layer Test 5 ---
xsdb% Opened binary file data/image_0_data/layer_4_output.bin
xsdb% Opened binary file data/image_0_data/layer_5_output.bin
xsdb% Timer Layer Naive: elapsed=1.000000ms
xsdb% Timer Layer SIMD: elapsed=1.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 5 timings (ms): naive=1.00, simd=1.00, speedup=1.00x
[Info]: --- Running Layer Test 6 ---
xsdb% Opened binary file data/image_0_data/layer_5_output.bin
xsdb% Opened binary file data/image_0_data/layer_6_output.bin
xsdb% Timer Layer Naive: elapsed=87.000000ms
xsdb% Timer Layer SIMD: elapsed=20.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 6 timings (ms): naive=87.00, simd=20.00, speedup=4.35x
[Info]: --- Running Layer Test 7 ---
xsdb% Opened binary file data/image_0_data/layer_6_output.bin
xsdb% Opened binary file data/image_0_data/layer_7_output.bin
xsdb% Timer Layer Naive: elapsed=112.000000ms
xsdb% Timer Layer SIMD: elapsed=30.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 7 timings (ms): naive=112.00, simd=30.00, speedup=3.73x
```

```
[Info]: --- Running Layer Test 8 ---
xsdb% Opened binary file data/image_0_data/layer_7_output.bin
xsdb% Opened binary file data/image_0_data/layer_8_output.bin
xsdb% Timer Layer Naive: elapsed=0.000000ms
xsdb% Timer Layer SIMD: elapsed=0.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 8 timings (ms): naive=0.00, simd=0.00, speedup=0.00x
[Info]: --- Running Layer Test 9 ---
xsdb% Opened binary file data/image_0_data/layer_8_output.bin
xsdb% Opened binary file data/image_0_data/layer_9_output.bin
xsdb% Timer Layer Naive: elapsed=0.000000ms
Timer Layer SIMD: elapsed=0.000000ms
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 9 timings (ms): naive=0.00, simd=0.00, speedup=0.00x
[Info]: --- Running Layer Test 10 ---
xsdb% Opened binary file data/image_0_data/layer_9_output.bin
xsdb% Opened binary file data/image_0_data/layer_10_output.bin
xsdb% Timer Layer Naive: elapsed=27.000000ms
xsdb% Timer Layer SIMD: elapsed=8.000000ms
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Layer 10 timings (ms): naive=27.00, simd=8.00, speedup=3.38x
[Info]: --- Running Layer Test 11 ---
xsdb% Opened binary file data/image_0_data/layer_10_output.bin
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Timer Layer Naive: elapsed=0.000000ms
xsdb% Timer Layer SIMD: elapsed=0.000000ms
Comparing Outputs (Cosine Similarity): False 0.0129771%  (0.000129771)
[Info]: Layer 11 timings (ms): naive=0.00, simd=0.00, speedup=0.00x
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=3014.000000ms
xsdb% Timer Full Inference TILED: elapsed=2179.000000ms
xsdb% Timer Full Inference SIMD: elapsed=783.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% [Info]: Full Inference Naive vs expected
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Full Inference TILED vs expected
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Full Inference SIMD vs expected
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: Full inference timings (ms): naive=3014.00, tiled=2179.00, simd=783.00, speedup=3.85x


----- ML::runTests() COMPLETE -----
```

- SIMD loads 4 weights at once into 128 bit register, and accumulates with single fused multiply-add (see below)
    - vmlaq_f32 on NEON
    - each lane does sum[i] += weight[i] * input in one instruction
- Across layers 0-8, SIMD reduces Zedboard total runtime from 2919 ms to 769
- Almost 4x speedup which makes sense as we set VECTOR_WIDTH to 4 diving our work into 4

- Big speedups to conv layers (3-4, 6-7)
- Mathematically equivalent – just parallelized

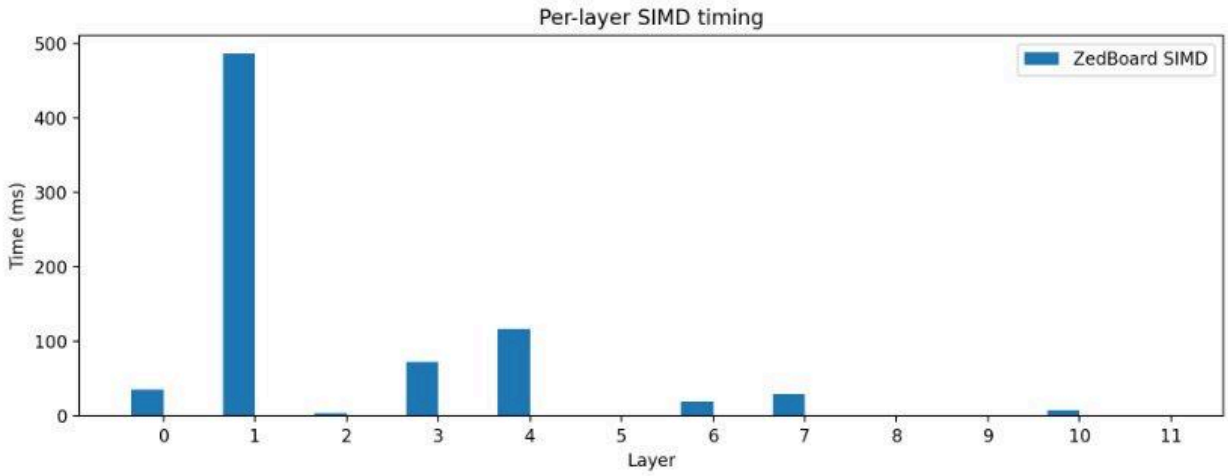
ARM NEON intrinsics

- Vdupq_n_f(x): duplicate a scalar float into all lanes of a 128-bit vector (q = 128-bit, n = scalar). Produces {x,x,x,x} for multiply/broadcast
- vmlaq_f32(a, b, c): vector multiply-accumulate, computes a + b * c
- ReLU with vmaxq_f32() and a zero vector

Remainder Handling

- Remainder inputs (if inputParams % 4 != 0) are handles by reverting back to naïve implementation – they aren't parallelized which can cause slowdowns (think if params 63 X 63 X 3)
- Interesting Idea: Variable VECTOR_WIDTH where you can experiment with different parallelization, adjusting itself per layer for maximum speedups

---



Layer	ZED Naïve Time (ms)	ZED SIMD Time (ms)	Speedup
0	101	36	2.81x
1	1854	487	3.81x

2	5	4	1.25x
3	288	73	3.95x
4	495	118	4.19x
5 -maxpool	1	1	Too small to matter
6	85	20	4.25x
7	90	30	3.23x
8-maxpool	<1	<1	Too small
9-flatten	<1	<1	Too small
10	27	8	3.38x
11- dense	1	1	n/a
Full	3012	784	3.84x

implements the vectorized kernel twice:

- an _mm_* version for x86 builds
- float32x4_t version for ARM/NEON

We vectorize across four output channels at a time (VECTOR_WIDTH = 4) while keeping the scalar loops over output (y,x) and over each filter position (r,s,c).
For each vector chunk

1. broadcast scalar input pixel (_mm_set1_ps / vdupq_n_f32)
2. load four filter weights
3. multiply-add-style accumulate (_mm_add_ps(... _mm_mul_ps(...)) / vmlaq_f32)
4. Add the bias vector
5. applies ReLU (max, zero vector)

Same as naive so we can reuse stride computations, but four channels per iteration dramatically lowers instruction count and better uses cache bandwidth.

Tail handling

When the output depth isn't a multiple of four, computeSIMD falls back to a scalar loop for the final channels (for (; m < outputDepth; ++m)),

Dense/other layers follow the same pattern: SIMD first, then a scalar cleanup pass